Open in app ↗

Medium Search Write

🚀 Angular 19 introduced true HMR improvements (big upgrade)

Piyali Das 4 min read · Apr 3, 2026

> 👉 *That means Angular dev server with HMR (Hot Module Reload) is active*
>
> 👉 *The **WebSocket (`client → websocket (Pending)`) is NOT new in Angular 19***
> 👉 *But **what it's doing (HMR behavior)*** has changed significantly in Angular 19

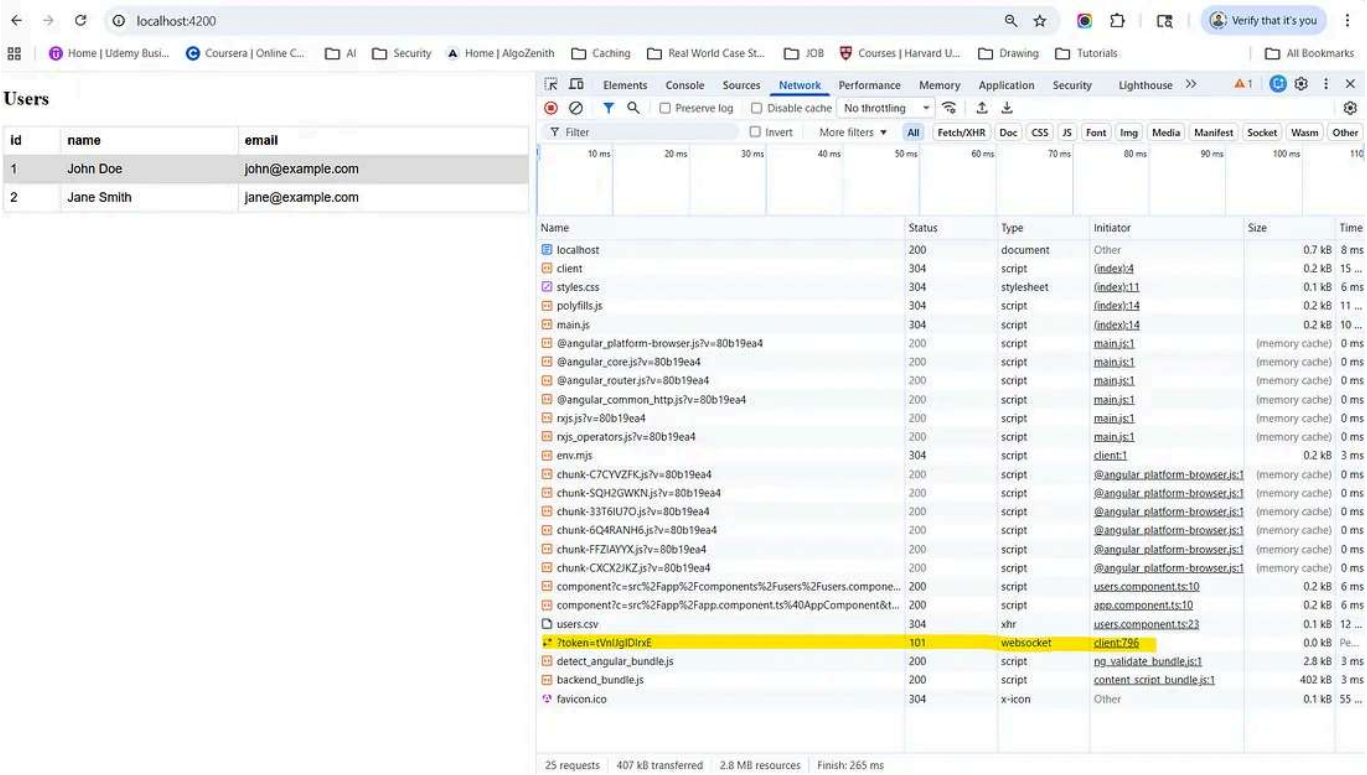
Angular 19 utilizes a fast dev server with default Hot Module Replacement (HMR) to update code changes without full page reloads. Upon saving, the CLI rebuilds, sends a WebSocket signal, and the browser hot-patches updated modules, CSS, or templates, improving developer experience by retaining application state.

- **HMR (Hot Module Replacement):** Styles and component templates in Angular 19 update instantly without a page refresh.
- **WebSocket Signals:** The Angular CLI (`ng serve`) communicates with the browser via WebSockets to trigger live reloads or HMR updates.

- **Websocket Resource Integration:** For real-time applications, WebSocket messages can be handled in Angular 19 using new resource signals to manage data streaming efficiently.
- **Troubleshooting Reload Loops:** If the app reloads continuously, it may be a module federation issue.
- **Customizing Reloads:** The `--live-reload=false` flag can be used with `ng serve` to disable automatic browser refreshes.

To try experimental HMR for templates, run:

```
NG_HMR_TEMPLATES=1 ng serve
```



🧠 1. WebSocket in Angular dev server (OLD behavior)

Even before Angular 19:

- `ng serve` always used a **dev server with WebSocket**

- Purpose:
- Live reload (refresh browser when code changes)

👉 Flow (old Angular):

```
Code change → Dev server → WebSocket signal → Browser reloads page
```

✓ Full page refresh
❌ State lost
❌ Slow for large apps

🚀 2. What changed in Angular 19

Angular 19 introduced **true HMR improvements (big upgrade)**:

✅ HMR is now enabled by default

- No need for `--hmr`
- Just:

```
ng serve
```

✅ Styles HMR (Angular 19.0)

- CSS updates → no reload
- Injected instantly

✅ Template HMR (Angular 19.1)

- HTML updates → no reload
- UI updates live

📌 **Source-backed:**

- Angular 19 enables HMR by default and patches app without reload
- Styles HMR is automatic; template HMR added in 19.1

🔌 3. So what is that WebSocket doing now?

Now the same WebSocket is used for **smarter updates**

New flow (Angular 19):

```
Code change → Dev server → WebSocket → Patch only changed part
```

Instead of:

```
→ Reload entire app ❌
```

⚠️ Important nuance (most people miss this)

❌ Angular 19 does NOT fully support HMR for everything

```
| File Type         | Behavior           |
| ---------------- | ----------------- |
| CSS / SCSS        | ✅ HMR (no reload) |
| HTML templates    | ✅ HMR             |
| TypeScript (.ts) | ❌ Full reload     |
```

👉 So if you change:

- `users.component.ts` → page reload
- `users.component.html` → instant update

🔥 4. Why you saw it in Network tab

This:

```
client → websocket (Pending)
```

Means:

✓ Dev server is connected
✓ HMR / live reload channel is active
✓ Angular is in **development mode**

🧪 5. How to verify HMR is working

Try this:

Change CSS:

```
h1 { color: red; }
```

👉 Updates instantly ✅

Change HTML:

```
<h1>Hello</h1>
```

👉 Updates instantly ✅

Change TS:

```
title = "New";
```

👉 Full reload 🔄

🛑 6. Want old behavior? (disable HMR)

```
ng serve --no-hmr
```

or in `angular.json`:

```
"hmr": false
```

✅ Final Answer

👉 WebSocket in dev tools is **NOT new**
👉 But **Angular 19 changed what it does**

Before Angular 19:

- WebSocket → triggers full reload

Angular 19+:

- WebSocket → enables **partial updates (HMR)**

💡 Real takeaway

> Angular finally moved closer to React/Vite-style dev experience 🚀
> But still **not full HMR (TS support missing)**

🧠 What "(memory cache)" means in DevTools

When you see:

```
Status: 200 (from memory cache)
```

👉 It means:

- The browser **did NOT make a network request**
- It served the file directly from **RAM (memory)**

🚀 Why you see so many memory cache entries

In your Angular app, this happens for 3 main reasons:

1. Angular Dev Server aggressively caches modules

Files like:

- `@angular_core.js`
- `rxjs.js`
- `chunk-XXXX.js`

👉 These are **ES modules**

Modern browsers:

- Load them once
- Keep them in memory
- Reuse them instantly

✓ Result → `(memory cache)`

2. Hot Reload / Live Reload (VERY important)

You have this:

```
client → websocket (Pending)
```

👉 That means **Angular dev server with HMR (Hot Module Reload)** is active

So what happens:

- Only changed files reload
- Everything else stays in memory

✓ That's why most files = memory cach

3. Browser optimization (performance boost)

Memory cache is the **fastest possible loading strategy**

```
| Source       | Speed     |
| ------------ | --------- |
| Memory cache | ⚡ Fastest |
| Disk cache   | Fast      |
| Network      | Slow      |
```

👉 Browser prefers memory cache whenever possible

✅ Final Answer

👉 You see many `(memory cache)` entries because:

✓ Angular uses module-based loading
✓ Dev server uses hot reload
✓ Browser optimizes performance
✓ Files are reused instead of re-downloaded

💡 **Bottom line:**

> This is expected and GOOD behavior — not an issue.

Angular | Technology | Coding | Front End Development

Written by Piyali Das

Edit profile

787 followers · 9 following

12 + yrs | Angular | NGRX | RxJS | https://www.linkedin.com/in/piyalidas10/
https://stackblitz.com/@piyalidas10 | https://github.com/piyalidas10

No responses yet

Piyali Das

What are your thoughts?